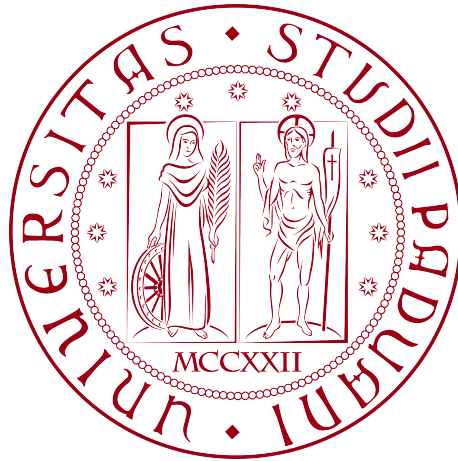


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



**Sviluppo di un Middleware basato su Spring per
l'Integrazione di Flussi di Dati**

Tesi di Laurea Triennale

Relatore
Prof. Zanella Marco

Laureando
Baldo Leonardo
Matricola 2042372

Ringraziamenti

Desidero esprimere la mia gratitudine al professor Zanella Marco, mio relatore, per l'aiuto e il sostegno che mi ha dato durante la stesura dell'elaborato.

Vorrei anche ringraziare, con affetto, i miei genitori per il loro sostegno, il grande aiuto e la loro presenza in ogni momento durante gli anni di studio.

Desidero poi ringraziare i miei amici per i bellissimi anni trascorsi insieme e le mille avventure vissute.

Padova, 28 luglio 2026

Baldo Leonardo

Abstract

Il presente documento descrive il lavoro svolto durante il periodo di stage, dalla durata di trecentoventi ore, dal laureando Baldo Leonardopresso l'azienda SyncLab S.r.l.

Il progetto di stage svolto riguarda lo studio e l'implementazione di *Middleware*, in particolar modo il *framework* Spring. L'applicazione da sviluppare dovrà leggere i dati da un Topic Kafka, arricchirli con dati presenti su un database e caricare il dato finale su un altro Topic Kafka.

Prima di iniziare con lo sviluppo effettivo del progetto è stato svolto uno studio approfondito delle tecnologie e degli strumenti coinvolti, i quali verranno analizzati nei successivi capitoli.

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Organizzazione del testo	1
1.2.1	Struttura del testo	1
1.2.2	Convenzioni tipografiche	2
2	Descrizione dello Stage	3
2.1	Introduzione al progetto	3
2.2	Organizzazione del lavoro	3
2.3	Analisi preventiva dei rischi	4
3	Analisi dei Requisiti	6
3.1	Introduzione	6
3.2	Obiettivi dello stage	6
3.3	Tracciamento dei requisiti	7
3.4	Tabella dei requisiti	8
4	Middleware	10
4.1	Introduzione al concetto di Middleware	10
4.2	Definizione e ruolo del Middleware	10
4.3	Classificazione dei Middleware	11
4.4	Enterprise Integration Patterns	11
4.4.1	Content Enricher	11
4.4.2	Dead Letter Channel	12
4.4.3	Altri pattern rilevanti	13
4.5	Apache Kafka come servizio di messaggistica	14
4.5.1	Introduzione e caratteristiche generali	14
4.5.2	Event streaming	14
4.5.3	Architettura e concetti fondamentali	14
4.5.4	Persistenza dei messaggi	16
4.5.5	Semantica di consegna	16
4.6	Spring come piattaforma middleware	17
4.6.1	Spring Boot	17
4.6.2	Spring Data JPA	18
4.6.3	Spring Kafka	18
5	Codifica	19
5.1	Tecnologie e strumenti	19
5.2	Progettazione	21
5.3	Implementazione	21
6	Conclusioni	22
	Acronimi	23

Elenco delle figure

1.1 Synclab S.r.l.	1
4.1 Content Enricher	12
4.2 Content Enricher	12
4.3 Message Channel	13
4.4 Point-to-Point Channel	13
4.5 Message Translator	13
4.6 Apache Kafka	16
5.1 Java	19
5.2 Spring	20
5.3 PostgreSQL	20
5.4 Apache Kafka	20
5.5 Docker	21
5.6 Apache Maven	21

Elenco delle tabelle

3.1 Tabella del tracciamento dei requisiti funzionali	8
3.2 Tabella del tracciamento dei requisiti qualitativi	8
3.3 Tabella del tracciamento dei requisiti di vincolo	9

Capitolo 1

Introduzione

1.1 L'azienda

L'esperienza di tirocinio svolta dal laureando ha avuto luogo negli uffici della sede veronese dell'azienda SyncLab S.r.l.¹

Sync Lab è una Innovative Company attenta ai paradigmi della trasformazione digitale che realizza prodotti e soluzioni per diversi mercati quali: Sanità, Industria, Energia, Telco, Finanza e Trasporti & Logistica.

L'offerta di consulenza specialistica, sintesi di circa 20 anni nel settore IT e la forte competenza in diversi domini tecnologici, aiutano il cliente a dominare i nuovi trend e le sfide che ne derivano, quali: GDPR, Big Data, Cloud Computing, IoT, Mobile e Cyber Security. In questo scenario Sync Lab si posiziona come uno dei più importanti system integrator nel panorama italiano delle aziende ICT.



Figura 1.1: Syncclab S.r.l.

1.2 Organizzazione del testo

1.2.1 Struttura del testo

Il presente documento è suddiviso in . . . capitoli il cui contenuto è brevemente riassunto in seguito:

Il secondo capitolo è composto da un'introduzione al progetto, dall'organizzazione del testo e dall'analisi preventiva dei rischi;

Il terzo capitolo è dedicato agli obiettivi e all'analisi dei requisiti;

Il quarto capitolo . . .

¹SyncLab S.r.l. - URL: <https://syncclab.it/home>

1.2.2 Convenzioni tipografiche

In merito alla redazione del presente documento, sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune menzionati vengono definiti nel glossario, situato alla fine del presente documento;
- i termini particolarmente rilevanti in una sezione e quelli in lingua straniera non di uso comune o facenti parti del gergo tecnico sono evidenziati con il carattere corsivo, fatta eccezione per le occorrenze presenti nei titoli delle sezioni o nelle didascalie;
- i comandi di terminale, i frammenti di codice sorgente e i nomi di file o directory sono evidenziati con il carattere monospaziato.

Capitolo 2

Descrizione dello Stage

2.1 Introduzione al progetto

Lo scopo principale di questo tirocinio sarà lo studio e l'implementazione di Middleware. Principalmente verrà sviluppata con il framework Spring (in particolare SpringBoot, Spring Data/JPA e SprintBatch), inoltre confrontarla con la piattaforma Apache Flink, se possibile.

L'applicazione da sviluppare dovrà leggere i dati da un Topic Kafka, arricchirli con dati presenti su PostgreSQL e caricare il dato finale su un altro Topic Kafka. Pertanto, il compito dello studente sarà quello di studiare, progettare, sviluppare e implementare tutte queste funzionalità sulle tecnologie sopra riportate.

2.2 Organizzazione del lavoro

Il lavoro è stato suddiviso in due fasi principali: una prima fase dedicata allo studio delle tecnologie e degli strumenti da utilizzare, in particolare del framework Spring, e una seconda fase dedicata all'implementazione vera e propria.

Il tirocinio si è svolto prevalentemente in modalità remota, con presenza in azienda una volta a settimana per confrontarmi con il tutor aziendale sull'andamento del progetto e per individuare eventuali criticità da correggere.

2.3 Analisi preventiva dei rischi

Per ogni rischio vengono indicati **descrizione** e **soluzione**, inoltre la **probabilità** che esso accada e l'**impatto** che il danno causerebbe.

1. Curva di apprendimento ripida

Probabilità: Alta (quasi certo che accada con stack così ampio).

Impatto: Medio (mitigabile con una fase di studio iniziale).

Descrizione: il progetto prevede l'utilizzo di tecnologie e strumenti di cui non si ha esperienza (Spring Boot, Spring Data/JPA, Spring Batch, Apache Kafka, PostgreSQL, eventualmente Apache Flink); questo può comportare rallentamenti dovuti allo studio e all'adozione delle nuove tecnologie, riducendo di conseguenza il tempo disponibile per lo sviluppo.

Soluzione: il periodo di tirocinio prevede una prima parte di studio delle tecnologie e degli strumenti che si andranno ad utilizzare, con particolare attenzione al framework Spring.

2. Ambiente di sviluppo

Probabilità: Media (problemi di setup sono comuni).

Impatto: Medio (risolvibili rapidamente con Docker).

Descrizione: è necessario un ambiente che includa Kafka, PostgreSQL e tutti gli strumenti utilizzati, configurato in modo da funzionare in modo affidabile e da rendere eventuali modifiche facilmente ripristinabili.

Soluzione: implementando il progetto tramite Docker, risulta più semplice e rapido il setup e l'integrazione di ogni strumento.

3. Integrazione tra tecnologie eterogenee

Probabilità: Alta (cuore tecnico del progetto).

Impatto: Alto (compromissione funzionalità principali).

Descrizione: il flusso dati tra Apache Kafka e PostgreSQL comporta problemi tipici dei sistemi distribuiti: gestione della concorrenza, consistenza dei dati, latenza nelle query di arricchimento e gestione dei fallimenti parziali.

Soluzione: definire una strategia di error handling.

4. Gestione dati inconsistenti o mancanti durante l'arricchimento

Probabilità: Media (prevedibile in sistemi di arricchimento dati).

Impatto: Medio (con scarto l'impatto resta contenuto).

Descrizione: può verificarsi il caso in cui il dato proveniente da Apache Kafka faccia riferimento a un record non presente in PostgreSQL; è quindi necessario definire una politica di gestione di questi casi.

Soluzione: in caso di dati errati o non trovati, questi vengono scartati e viene generata una notifica tramite messaggio di errore.

5. Confronto con Apache Flink

Probabilità: Alta (molto probabile per vincoli di tempo).

Impatto: Basso (opzionale, quindi di impatto contenuto).

Descrizione: Flink adotta un modello di programmazione molto diverso da Spring; il confronto tra le due tecnologie richiede una comprensione approfondita di entrambi i paradigmi per risultare significativo e non banale.

Soluzione: tale attività viene indicata come obiettivo desiderabile e non obbligatorio.

Capitolo 3

Analisi dei Requisiti

3.1 Introduzione

L'analisi dei requisiti è un'attività preliminare allo sviluppo di un sistema software, il cui scopo è quello di definire le funzionalità che il nuovo prodotto deve offrire, ovvero i requisiti che devono essere soddisfatti dal software sviluppato. L'analisi dei requisiti è di fondamentale importanza, in quanto essa guida le successive fasi di sviluppo del prodotto.

3.2 Obiettivi dello stage

Notazione

Si farà riferimento ai requisiti secondo le seguenti notazioni:

- **O** per i requisiti obbligatori, vincolanti in quanto obiettivo primario richiesto dal committente;
- **D** per i requisiti desiderabili, non coinvolanti o strettamente necessari, ma dal riconoscibile valore aggiunto;
- **F** per i requisiti facoltativi, rappresentanti valore aggiunto non strettamente competitivo.

Le sigle precedentemente indicate saranno seguite da una coppia sequenziale di numeri, identificativo del requisito.

Obiettivi fissati

Si prevede lo svolgimento dei seguenti obiettivi:

- Obbligatori
 - **O01**: Studio framework Spring;
 - **O02**: Implementazione flusso mediante Spring;
 - **O03**: Relazione tecnica che descriva le scelte tecniche prese durante il percorso e le motivazioni a riguardo.
- Desiderabili
 - **D01**: Studio Apache Flink;
 - **D02**: Implementazione flusso mediante Apache Flink;
 - **D03**: Affrontare un'analisi comparativa usando metriche ben precise e coerenti con il contesto applicativo.
- Facoltativi
 - **F01**: Effettuare dei test di performance mediante strumenti di misurazione dedicati;
 - **F02**: Implementare il flusso anche su tecnologie alternative (Apache Camel, Apache NiFi).

3.3 Tracciamento dei requisiti

Da un'analisi degli obiettivi e dei requisiti effettuata sul progetto è stata stilata la tabella che traccia i requisiti. Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli. Il codice dei requisiti, dove ogni requisito è identificato con il carattere **R**, è strutturato nel seguente modo:

R[tipologia][importanza]-[numero]

In particolare le tipologie sono:

- **F** per i requisiti funzionali;
- **Q** per i requisiti qualitativi;
- **V** per i requisiti di vincolo.

In particolare le importanze sono:

- **O** per i requisiti obbligatori;
- **D** per i requisiti desiderabili;
- **F** per i requisiti facoltativi.

3.4 Tabella dei requisiti

Requisito	Descrizione	Stato
RFO-01	Il sistema deve leggere (consumare) messaggi da un Topic Kafka sorgente	Soddisfatto
RFO-02	Il sistema deve interrogare un database per recuperare dati di arricchimento correlati al messaggio ricevuto	Soddisfatto
RFO-03	Il sistema deve combinare (arricchire) il dato originale ricevuto da Apache Kafka con i dati letti dal database	Soddisfatto
RFO-04	Il sistema deve pubblicare (produrre) il dato arricchito su un Topic Kafka di destinazione, diverso da quello sorgente	Soddisfatto
RFO-05	Il sistema deve gestire la connessione e configurazione verso Apache Kafka (consumer e producer) tramite Spring	Soddisfatto
RFD-06	Il sistema deve gestire gli errori di elaborazione, come dato non trovato sul database, prevedendo una strategia	Soddisfatto
RFD-07	Deve essere realizzata un'implementazione equivalente delle stesse funzionalità utilizzando Apache Flink, per permettere il confronto	Insoddisfatto

Tabella 3.1: Tabella del tracciamento dei requisiti funzionali

Requisito	Descrizione	Stato
RQD-01	Il codice deve essere organizzato in modo modulare, con separazione tra strato di ingestion, arricchimento e produzione, per favorire manutenzione e test	Soddisfatto
RQD-02	Le due implementazioni (Spring vs Flink) devono essere progettate in modo da poter essere confrontate su metriche omogenee	Insoddisfatto

Tabella 3.2: Tabella del tracciamento dei requisiti qualitativi

Requisito	Descrizione	Stato
RVO-01	Deve essere utilizzato il framework Spring	Soddisfatto
RVO-02	La sorgente dati di input è vincolata a essere un Topic Kafka	Soddisfatto
RVO-03	L'output deve essere vincolato a un Topic Kafka di destinazione (diverso da quello di input)	Soddisfatto

Tabella 3.3: Tabella del tracciamento dei requisiti di vincolo

Capitolo 4

Middleware

4.1 Introduzione al concetto di Middleware

Nell'ambito dei sistemi informativi moderni, la crescente necessità di integrare applicazioni eterogenee, spesso sviluppate con tecnologie diverse e distribuite su più nodi, ha reso centrale il ruolo del middleware. Con questo termine si indica genericamente quello strato software che si colloca tra il sistema operativo e le applicazioni, con lo scopo di semplificare la comunicazione, la gestione dei dati e l'interoperabilità tra componenti distinte di un sistema distribuito.

Il presente tirocinio si colloca proprio in questo ambito: l'obiettivo è studiare, progettare e implementare un middleware in grado di leggere dati da un topic Kafka, arricchirli con informazioni provenienti da un database relazionale PostgreSQL, e pubblicare il risultato su un secondo topic Kafka. Questo capitolo introduce i concetti teorici alla base del middleware in generale, per poi restringere il campo alle tecnologie specifiche impiegate nel progetto: Spring Boot, Spring Data JPA e Apache Kafka.

4.2 Definizione e ruolo del Middleware

Il middleware è un tipo di programma software che fornisce alle applicazioni software servizi che vanno oltre quelli offerti dal sistema operativo. È lo strato intermedio tra le applicazioni e le componenti sottostanti, come sistemi operativi o database, facilitando così la comunicazione, l'integrazione e la gestione delle risorse nei sistemi distribuiti. Il suo ruolo primario è quello di nascondere la complessità dell'infrastruttura sottostante, garantendo interoperabilità, scalabilità e sicurezza.

Le caratteristiche principali sono:

- **Disaccoppiamento:** i sistemi produttori e consumatori di informazioni non devono obbligatoriamente conoscersi o condividere lo stesso linguaggio, piattaforma o formato dati.
- **Interoperabilità:** consente la comunicazione tra applicazioni scritte in linguaggi diversi o eseguite su piattaforme differenti.
- **Scalabilità e affidabilità:** molti middleware offrono tolleranza ai guasti e garanzie di consegna dei messaggi.

Il middleware sviluppato in questo progetto ha la funzione di ponte tra due flussi di eventi Kafka, arricchendo il dato in transito con informazioni

recuperate da PostgreSQL. Si tratta quindi di un middleware orientato ai dati e agli eventi, piuttosto che di un semplice bus di comunicazione RPC.

4.3 Classificazione dei Middleware

Sono state definite numerose categorie di middleware, in base al settore in cui vengono impiegate o al modulo applicativo che servono. Una bibliografia recente ha identificato le principali categorie di middleware come segue:

- **Transactional Middleware:** elaborazione di transazioni multiple sincrone/asincrone, fungendo da cluster di richieste associate provenienti da sistemi distribuiti (es. transazioni bancarie o pagamenti con carta di credito).
- **Message-Oriented Middleware (MOM):** basati sullo scambio di messaggi asincroni attraverso code o topic (es. Apache Kafka).
- **Procedural Middleware:** architetture remote e locali per connettersi, trasmettere e recuperare le risposte software delle comunicazioni tra sistemi asincroni (operazione di chiamata).
- **Object-Oriented Middleware:** simile al middleware procedurale, ma incorpora i principi di progettazione della programmazione orientata agli oggetti. Dal punto di vista analitico, i suoi componenti software comprendono riferimenti agli oggetti, eccezioni ed ereditarietà delle proprietà tramite richieste di oggetti distribuiti.

Il progetto oggetto del tirocinio si colloca nella seconda categoria: si tratta infatti di un Message-Oriented Middleware con funzionalità di arricchimento dati (data enrichment), in cui il canale di comunicazione è rappresentato da Kafka e la logica di trasformazione è delegata a un componente applicativo (Spring).

4.4 Enterprise Integration Patterns

Per inquadrare correttamente il problema dal punto di vista architetturale, è utile fare riferimento agli Enterprise Integration Patterns (EIP), un catalogo di pattern ricorrenti nell'integrazione di sistemi enterprise, che permette di descrivere il problema del progetto in termini standard e riconosciuti in letteratura.

4.4.1 Content Enricher

Il pattern più rilevante per questo lavoro è il Content Enricher: un componente che riceve un messaggio contenente informazioni parziali e lo arricchisce con dati aggiuntivi provenienti da una fonte esterna (database relazionale), producendo un messaggio più completo da inoltrare al passo successivo della pipeline.

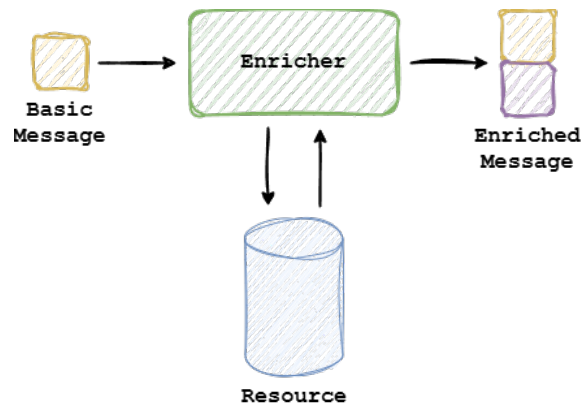


Figura 4.1: Content Enricher

Nell'implementazione sviluppata, questo pattern è realizzato concretamente dalla classe `EnrichmentService`, che riceve in input un `OrderEvent` (contenente, tra gli altri campi, un `customerId`), interroga PostgreSQL tramite `CustomerRepository` (Spring Data JPA) per recuperare i dati del cliente corrispondente, e produce in output un `EnrichedOrder`, ovvero il messaggio arricchito con le informazioni mancanti.

4.4.2 Dead Letter Channel

Un secondo pattern, altrettanto rilevante, è il Dead Letter Channel. Questo pattern affronta un problema tipico dei sistemi di integrazione basati su messaggi: cosa fare quando un messaggio non può essere elaborato correttamente? Le opzioni alternative sono entrambe insoddisfacenti:

- Bloccare la pipeline, in attesa che il problema venga risolto manualmente, con il rischio di fermare l'elaborazione di tutti i messaggi successivi (in Kafka, il blocco su una partizione impedisce l'avanzamento dell'offset per l'intera partizione);
- Scartare silenziosamente il messaggio, con conseguente perdita di dati e nessuna possibilità di analisi o recupero successivo.

Il pattern Dead Letter Channel propone invece una terza via: i messaggi che non possono essere elaborati vengono instradati su un canale separato (la dead letter queue, o DLQ), dedicato esclusivamente alla raccolta dei messaggi "falliti". Questo consente al flusso principale di proseguire senza interruzioni, mantenendo al contempo traccia dei messaggi problematici per un'analisi o un riprocessamento successivo.

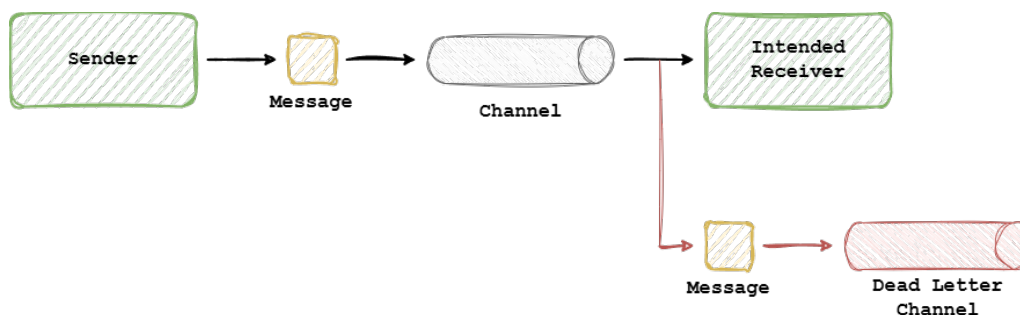


Figura 4.2: Content Enricher

Nel progetto, questo pattern è implementato nel modo seguente: quando `EnrichmentService` tenta di arricchire un `OrderEvent` con un `customerId` non presente su PostgreSQL, viene sollevata una `CustomerNotFoundException`. Questa eccezione viene intercettata e gestita instradando il messaggio originale sul topic `orders-input-dlq`, anziché proseguire verso `EnrichedProducer` e il topic `orders-output`. In questo modo, un singolo dato incompleto o malformato non compromette l'elaborazione degli ordini successivi, e resta comunque disponibile per un'ispezione a posteriori (ad esempio tramite la Kafka UI predisposta nell'ambiente di sviluppo).

4.4.3 Altri pattern rilevanti

Altri pattern EIP utili e applicabili al progetto sono:

- **Message Channel:** collega le applicazioni tramite un canale di messaggi, in cui un'applicazione scrive le informazioni nel canale e l'altra le legge dal canale. I messaggi qui rappresentato dai topic Kafka (`orders-input`, `orders-output`, `orders-input-dlq`).

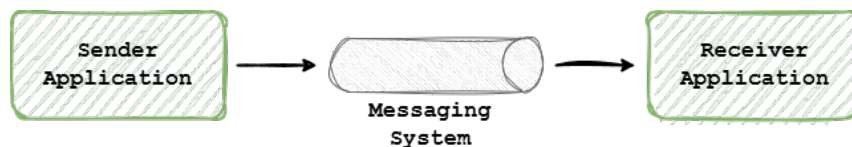


Figura 4.3: Message Channel

- **Point-to-Point Channel:** invia il messaggio su un canale Point-to-Point, che garantisce che solo un destinatario riceva un determinato messaggio.

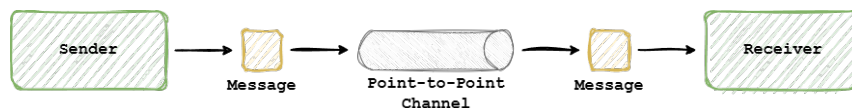


Figura 4.4: Point-to-Point Channel

- **Message Translator:** utilizza un filtro speciale (Translator), tra altri filtri o applicazioni per convertire un formato di dati in un altro. Nel progetto è usato nella conversione tra `OrderEvent` e `EnrichedOrder`, entrambi DTO distinti nel package `dto/`.

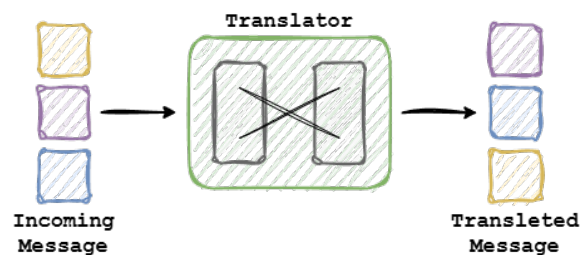


Figura 4.5: Message Translator

4.5 Apache Kafka come servizio di messaggistica

4.5.1 Introduzione e caratteristiche generali

Apache Kafka è una piattaforma distribuita per lo streaming di eventi, originariamente sviluppata da LinkedIn¹ e successivamente rilasciata come progetto open source sviluppata da Apache Software Foundation².

A differenza dei tradizionali sistemi di messaggistica basati su code (come i classici Message Queue), Kafka nasce con l'obiettivo di gestire flussi di dati ad alto *throughput*, bassa latenza e con garanzie di persistenza e ripetibilità della lettura, caratteristiche che lo rendono particolarmente adatto sia a scenari di semplice trasporto di messaggi, sia a scenari più complessi di *event streaming* ed *event sourcing*.

Nel contesto di questo progetto, Kafka non è impiegato semplicemente come canale di trasporto, ma come componente architetturale a tutti gli effetti: da esso dipendono sia l'ingresso dei dati grezzi nel sistema, sia l'uscita dei dati arricchiti.

4.5.2 Event streaming

L'*event streaming* è l'equivalente digitale del sistema nervoso centrale del corpo umano. Rappresenta il fondamento tecnologico di un mondo "sempre attivo", in cui le aziende sono sempre più definite dal *software* e automatizzate, e in cui l'utente del *software* è a sua volta un *software*.

Dal punto di vista tecnico, lo streaming di eventi consiste in:

- acquisizione in tempo reale di dati provenienti da fonti quali *database*, sensori, dispositivi mobili, servizi cloud e applicazioni software sotto forma di flussi di eventi;
- archiviazione duratura di tali flussi per un successivo recupero; nella manipolazione, elaborazione e risposta ai flussi di eventi sia in tempo reale che a posteriori;
- instradamento dei flussi di eventi verso diverse tecnologie di destinazione a seconda delle necessità.

L'*event streaming* garantisce quindi un flusso continuo e l'interpretazione dei dati, in modo che le informazioni giuste si trovino nel posto giusto, al momento giusto.

4.5.3 Architettura e concetti fondamentali

Producer e Consumer I produttori sono quelle applicazioni client che pubblicano (scrivono) eventi su Kafka, mentre i consumatori sono quelle che si abbonano (leggono ed elaborano) a tali eventi. In Kafka, produttori e consumatori sono completamente disaccoppiati e indipendenti l'uno dall'altro, il che costituisce un elemento chiave del design per ottenere l'elevata scalabilità (i produttori non devono mai attendere i consumatori).

Nel middleware sviluppato, l'applicazione stessa riveste entrambi i ruoli: agisce come consumer rispetto al topic sorgente (da cui legge i dati grezzi da arricchire) e come producer rispetto al topic di destinazione (su cui scrive il dato arricchito).

¹LinkedIn - URL: <https://www.linkedin.com>

²Apache Software Foundation - URL: <https://www.apache.org/>

Consumer Group Kafka introduce il concetto di consumer group, un insieme di consumer identificati da uno stesso `group.id`, che si ripartiscono automaticamente le partizioni di un topic, così che ciascun messaggio venga elaborato da un solo consumer all'interno del gruppo. Questo meccanismo consente di scalare orizzontalmente il middleware, aggiungendo istanze aggiuntive dell'applicazione senza necessità di logica di bilanciamento del carico esplicita: sarà Kafka stesso a ridistribuire le partizioni tra i consumer disponibili (fenomeno noto come *rebalance*).

Event Gli eventi vengono organizzati e archiviati in modo permanente in argomenti (topic). Un argomento è simile a una cartella in un file system, mentre gli eventi sono i file contenuti in quella cartella.

Gli argomenti in Kafka sono sempre multi-producer e multi-consumer : un argomento può avere zero, uno o più produttori che vi scrivono eventi, così come zero, uno o più consumatori che si abbonano a tali eventi. Gli eventi in un argomento possono essere letti tutte le volte che è necessario: a differenza dei sistemi di messaggistica tradizionali, gli eventi non vengono eliminati dopo la lettura. È l'utente a definire per quanto tempo Kafka debba conservare gli eventi tramite un'impostazione di configurazione specifica per ogni argomento, trascorso il quale gli eventi più vecchi verranno eliminati. Le prestazioni di Kafka rimangono sostanzialmente costanti indipendentemente dalla dimensione dei dati, quindi l'archiviazione dei dati per lunghi periodi di tempo non presenta alcun problema.

Topic L'unità logica fondamentale di Kafka è il topic, un canale a cui i producer inviano messaggi e da cui i consumer li leggono.

Ogni topic è fisicamente suddiviso in una o più partition, che rappresentano l'unità di parallelismo: ogni partizione è una sequenza ordinata e immutabile di messaggi, identificati da un numero progressivo detto *offset*. Questa distribuzione dei dati è molto importante per la scalabilità, poiché consente alle applicazioni client di leggere e scrivere i dati da e verso molti broker contemporaneamente. Quando un nuovo evento viene pubblicato su un argomento, viene effettivamente aggiunto a una delle partizioni dell'argomento.

Per garantire la tolleranza ai guasti e l'alta disponibilità dei dati, ogni argomento può essere replicato, anche tra diverse aree geografiche o data center, in modo che ci siano sempre più broker in possesso di una copia dei dati nel caso in cui si verificano problemi, si desideri eseguire operazioni di manutenzione sui broker e così via. Un'impostazione comune in produzione è un fattore di replica pari a 3, ovvero ci saranno sempre tre copie dei dati. Questa replica viene eseguita a livello di partizioni degli argomenti.

Broker e Cluster Un cluster Kafka è composto da uno o più broker, ciascuno responsabile della gestione di un sottoinsieme di partizioni. La distribuzione dei dati su più broker, unita alla replica delle partizioni (controllata dal parametro *replication factor*), garantisce tolleranza ai guasti: in caso di indisponibilità di un broker, un'altra replica può assumere il ruolo di leader per la partizione interessata, senza perdita di dati.

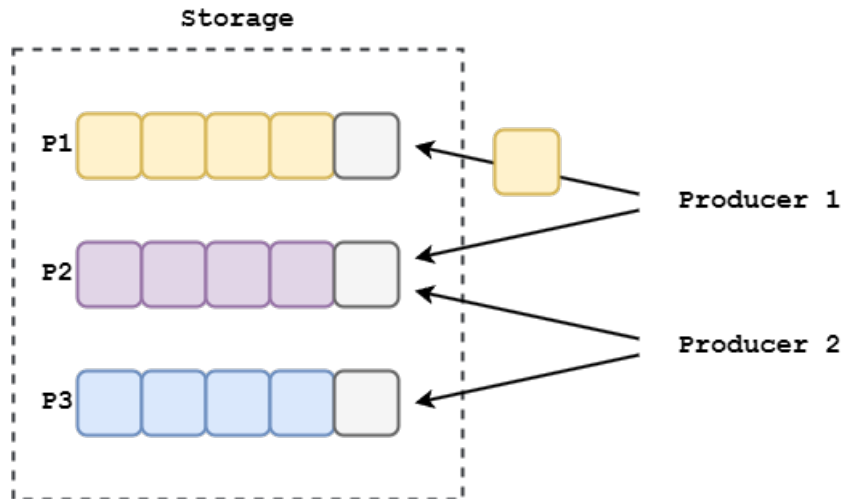


Figura 4.6: Apache Kafka

4.5.4 Persistenza dei messaggi

Una delle caratteristiche distintive di Kafka rispetto ai tradizionali MOM è la persistenza dei messaggi su disco, per un periodo di retention configurabile, indipendentemente dal fatto che siano già stati letti da un consumer.

Questo consente di:

- rileggere lo storico dei messaggi in caso di errore o necessità di ri-processamento;
- gestire il disaccoppiamento temporale tra producer e consumer, che non devono essere necessariamente attivi nello stesso momento;
- garantire resilienza del middleware in caso di malfunzionamento temporaneo della componente di arricchimento (indisponibilità del database), poiché i messaggi restano disponibili nel topic finché non vengono correttamente processati.

4.5.5 Semantica di consegna

Dal punto di vista della semantica di consegna, Kafka supporta tre modalità:

- **At-most-once**: il messaggio può essere perso, ma non elaborato più volte (offset committato prima dell'elaborazione).
- **At-least-once**: il messaggio viene sempre elaborato, ma può essere elaborato più volte in caso di errore o riavvio (offset committato dopo l'elaborazione). È la modalità più comune nei sistemi che, come quello del progetto, non tollerano perdita di dati.
- **Exactly-once**: garantisce che ogni messaggio venga elaborato esattamente una volta, tramite meccanismi di transazioni Kafka.

Nel contesto del middleware sviluppato, la scelta della semantica di consegna è un aspetto progettuale rilevante: poiché la fase di arricchimento comporta un'interrogazione al database PostgreSQL, occorre valutare attentamente le implicazioni di un'eventuale elaborazione duplicata del medesimo messaggio.

4.6 Spring come piattaforma middleware

Spring è tra i framework più diffusi in ambito enterprise per la costruzione di applicazioni Java robuste, modulari e facilmente integrabili con infrastrutture di messaggistica e basi di dati.

4.6.1 Spring Boot

Spring Boot rappresenta l'evoluzione del framework Spring pensata per semplificare la configurazione e l'avvio di applicazioni Java, tramite il paradigma dell'auto-configuration e della convention over configuration.

Alla base del framework resta il principio di *Inversion of Control* (IoC), realizzato tramite *Dependency Injection*: i componenti applicativi non istanziano direttamente le proprie dipendenze, ma le ricevono da un contenitore (l'application context), che ne gestisce il ciclo di vita.

Ai fini della realizzazione di un middleware, questo approccio offre alcuni vantaggi rilevanti:

- disaccoppiamento tra i vari componenti logici del sistema (lettura dei messaggi, logica di arricchimento, scrittura dei messaggi), che possono essere sviluppati, testati e sostituiti in modo indipendente;
- integrazione facile e veloce con le principali infrastrutture enterprise (broker di messaggistica, basi di dati, sistemi di monitoraggio), grazie a un ricco ecosistema di starter e librerie ufficiali;
- riduzione del codice infrastrutturale a carico dello sviluppatore, che può concentrarsi sulla logica di dominio.

Auto-configuration

La configurazione automatica di Spring Boot è una funzionalità che configura automaticamente i componenti dell'applicazione in base alle dipendenze disponibili.

Crea e configura automaticamente i servizi e le impostazioni necessari per l'applicazione. Riduce la configurazione manuale e accelera lo sviluppo.

La configurazione automatica funziona tramite l'annotazione `@EnableAutoConfiguration`.

Convention over configuration

Il principio *Convention over configuration* (noto anche come *coding by convention*) è un paradigma di progettazione software utilizzato dai framework che mira a ridurre il numero di decisioni che uno sviluppatore che utilizza il framework è tenuto a prendere, senza necessariamente rinunciare alla flessibilità e ai principi del *Don't Repeat Yourself* (DRY).

La frase significa essenzialmente che uno sviluppatore deve specificare solo gli aspetti non convenzionali dell'applicazione. Quando la convenzione implementata dallo strumento corrisponde al comportamento desiderato, il sistema si comporta come previsto senza dover scrivere file di configurazione. Solo quando il comportamento desiderato si discosta dalla convenzione implementata è necessaria una configurazione esplicita.

4.6.2 Spring Data JPA

Spring Data JPA è il modulo dell'ecosistema Spring dedicato alla persistenza dei dati, costruito sopra la specifica JPA (Java Persistence API) e sull'implementazione di riferimento Hibernate.

Il suo contributo principale, dal punto di vista architetturale, è l'astrazione dell'accesso ai dati tramite il pattern Repository. Lo sviluppatore definisce un'interfaccia che descrive le operazioni di accesso desiderate, senza doverne scrivere l'implementazione e le query SQL, che vengono generate automaticamente dal framework a partire dalle convenzioni di naming o da annotazioni dedicate.

Nel contesto di un middleware di enrichment, questo modulo assume un ruolo centrale: è il componente che permette di interrogare la fonte dati esterna (PostgreSQL) necessaria a completare le informazioni presenti nel messaggio in transito, disaccoppiando la logica di arricchimento dal dettaglio implementativo dell'accesso al database.

4.6.3 Spring Kafka

Per la parte di integrazione con Apache Kafka, l'ecosistema Spring mette a disposizione Spring Kafka, un modulo che fornisce un'astrazione ad alto livello sopra il client Kafka nativo, semplificando la configurazione dei producer e dei consumer e integrandoli pienamente con il modello di *dependency injection* tipico di Spring Boot.

Il concetto centrale su cui si fonda Spring Kafka è quello di *listener*: un componente applicativo si registra come consumatore di uno o più topic e viene invocato automaticamente ogni volta che un nuovo messaggio è disponibile, senza che lo sviluppatore lo debba gestire. Il modello di elaborazione risultante è quindi *event-driven* e continuo: l'applicazione resta costantemente in ascolto, reagendo agli eventi non appena questi si presentano, senza necessità di una schedulazione periodica.

Questo paradigma si adatta molto bene a scenari come quello affrontato nel presente lavoro, in cui il flusso di dati non ha una fine definita a priori (uno stream potenzialmente illimitato di eventi) e in cui è desiderabile minimizzare la latenza tra la produzione di un evento e la sua elaborazione. Spring Kafka realizza, a livello applicativo, le astrazioni Kafka già introdotte nella sezione 4.5.3 (topic, consumer group, offset).

Questi elementi rendono Spring Kafka una scelta architetturale coerente con il pattern Content Enricher (descritto in 4.4.1): il *listener* funge da punto di ingresso del messaggio grezzo, delega la logica di arricchimento a un componente dedicato (tipicamente interfacciato con la base dati tramite Spring Data JPA), e infine si occupa della pubblicazione del messaggio arricchito sul topic di destinazione, chiudendo così il ciclo di elaborazione descritto nello schema architetturale generale del middleware.

Capitolo 5

Codifica

5.1 Tecnologie e strumenti

Java

Java è un linguaggio di programmazione ad alto livello, orientato agli oggetti e a tipizzazione statica, che si appoggia sull'omonima piattaforma software di esecuzione, specificamente progettato per essere il più possibile indipendente dalla piattaforma hardware di esecuzione (tramite compilazione in bytecode prima e interpretazione poi da parte di una JVM).



Figura 5.1: Java

Spring

Spring è un framework Open Source per lo sviluppo di applicazioni su piattaforma Java. A questo framework sono associati tanti altri progetti, che hanno nomi composti come Spring Boot, Spring Data, Spring Batch, etc. Questi progetti sono stati ideati per fornire funzionalità aggiuntive al framework.

- Spring Boot
- Spring Data JPA
- Spring Cloud Stream



Figura 5.2: Spring

PostgreSQL

PostgreSQL è un completo DBMS ad oggetti pubblicato con licenza libera (stile Licenza BSD). Spesso abbreviato come "Postgres", sebbene questo sia un nome vecchio dello stesso progetto, è una reale alternativa sia rispetto ad altri prodotti liberi come MySQL, Firebird SQL e MaxDB che a quelli a codice chiuso come Oracle, IBM Informix o DB2 ed offre caratteristiche uniche nel suo genere che lo pongono per alcuni aspetti all'avanguardia nel settore delle basi di dati.



Figura 5.3: PostgreSQL

Apache Kafka

Apache Kafka è una piattaforma Open Source di stream processing scritta in Java e Scala e sviluppata dall'Apache Software Foundation. Il progetto mira a creare una piattaforma a bassa latenza ed alta velocità per la gestione di feed dati in tempo reale.

Questo progetto viene usato principalmente per tutte le applicazioni di elaborazioni di stream di dati in tempo reale.



Figura 5.4: Apache Kafka

Docker

Docker è un popolare software libero che utilizza la virtualizzazione a livello di sistema operativo per eseguire applicazioni in ambienti isolati chiamati container.

I container Docker sono un'estensione dei container del sistema operativo.

Docker supporta sia i container Linux che quelli Windows. È possibile utilizzare Docker con container Linux anche nei sistemi Windows e MacOS grazie a tecniche di virtualizzazione.



Figura 5.5: Docker

Maven

Apache Maven è uno strumento di gestione di progetti software basati su Java e build automation.

Maven usa un costrutto conosciuto come POM; un file XML che descrive le dipendenze fra il progetto e le varie versioni di librerie necessarie nonché le dipendenze fra di esse. In questo modo si separano le librerie dalla directory di progetto utilizzando questo file descrittivo per definirne le relazioni.

Maven effettua automaticamente il download di librerie Java e plug-in Maven dai vari repository definiti scaricandoli in locale o in un repository centralizzato lato sviluppo. Questo permette di recuperare in modo uniforme i vari file JAR e di poter spostare il progetto indipendentemente da un ambiente all'altro avendo la sicurezza di utilizzare sempre le stesse versioni delle librerie.



Figura 5.6: Apache Maven

5.2 Progettazione

...

5.3 Implementazione

...

Capitolo 6

Conclusioni

Acronimi

DBMS Database Management System. 20,
DLQ Dead Letter Queue. 12,
DTO Data Transfer Object. 13,
EIP Enterprise Integration Patterns. 11, 13,
IoT Internet of Things. 1,
JAR Java Archive. 21,
JVM Java Virtual Machine. 19,
MOM Message-Oriented Middleware. 11,
POM Project Object Model. 21,
RPC Remote Procedure Call. 11,
XML eXtensible Markup Language. 21,

Glossario

Middleware Il middleware è un software che funge da strato intermedio tra le applicazioni e le componenti sottostanti, come ad esempio sistemi operativi, database o hardware, facilitando la comunicazione, l'integrazione e la gestione delle risorse nei sistemi distribuiti. Il suo ruolo primario è quello di nascondere la complessità dell'infrastruttura sottostante, garantendo interoperabilità, scalabilità e sicurezza. iv, 3

Open Source software distribuito, generalmente in via gratuita, sotto i termini di una licenza open source. 19, 20

Topic Kafka I Topic di Kafka sono le categorie utilizzate per organizzare i messaggi. Ogni topic ha un nome unico per l'intero cluster Kafka. I messaggi vengono inviati e letti da argomenti specifici. I produttori scrivono dati negli argomenti e i consumatori leggono dati dagli argomenti. In Kafka, gli argomenti sono partizionati e replicati attraverso i broker per tutta l'implementazione. I broker si riferiscono a ciascuno dei nodi di un cluster Kafka. Le partizioni sono importanti perché permettono la parallelizzazione degli argomenti, consentendo un elevato throughput dei messaggi. iv, 3, 8, 9

Visual Studio Code editor di codice sorgente sviluppato da Microsoft per Windows, Linux e macOS. È un software libero e gratuito per uso personale e commerciale, anche se la versione ufficiale è sotto una licenza proprietaria. Permette di installare estensioni che aggiungono ulteriori funzionalità.

Bibliografia

Testi

...

Articoli

...

Siti web

Apache Kafka - URL: <https://kafka.apache.org/> - 4.5

Content Enricher - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/DataEnricher.html> - 4.4.1

Dead Letter Channel - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/DeadLetterChannel.html> - 4.4.2

Enterprise Integration Patterns - URL: <https://www.enterpriseintegrationpatterns.com/> - 4.4

Message Channel - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/MessageChannel.html> - 4.4.3

Message Translator - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/MessageTranslator.html> - 4.4.3

Middleware - URL: <https://en.wikipedia.org/wiki/Middleware> - 4

Middleware - URL: [https://en.wikipedia.org/wiki/Middleware_\(distributed_applications\)](https://en.wikipedia.org/wiki/Middleware_(distributed_applications)) - 4

Point-to-Point Channel - URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/PointToPointChannel.html> - 4.4.3

Spring Boot - URL: <https://spring.io/projects/spring-boot> - 4.6.1

Spring Boot - URL: https://en.wikipedia.org/wiki/Spring_Boot - 4.6.1

Spring Data JPA - URL: <https://spring.io/projects/spring-data-jpa> - 4.6.2

Spring Framework - URL: https://en.wikipedia.org/wiki/Spring_Framework - 4.6

Spring Kafka - URL: <https://spring.io/projects/spring-kafka> - 4.6.3